

Actividad 2: Análisis de Seguridad Dinámico con DAST (*Dynamic Application Security Testing*)

Tema: Escaneo de vulnerabilidades en aplicaciones en ejecución



Objetivo: Usar DAST para detectar vulnerabilidades en una aplicación en ejecución

¿Qué es DAST?

Dynamic Application Security Testing (DAST) es una metodología de análisis de seguridad que evalúa una aplicación en ejecución para identificar vulnerabilidades explotables. A diferencia de las pruebas estáticas (SAST), DAST no requiere acceso al código fuente y se enfoca en detectar problemas de seguridad en tiempo de ejecución, como inyecciones SQL, configuraciones inseguras y exposiciones de información sensible.

Nikto

Nikto es un escáner de seguridad de aplicaciones web de código abierto que detecta vulnerabilidades en servidores web y aplicaciones en ejecución.

Instalar Nikto para análisis DAST

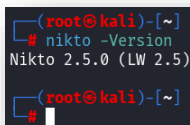
Ejecutar el siguiente comando en la terminal para instalar Nikto:

```
sudo apt install nikto
```

Verificar la instalación

Después de instalar Nikto, verificar que está funcionando correctamente:

```
nikto -Version
```



```
(root@kali)~[~]  
# nikto -Version  
Nikto 2.5.0 (LW 2.5)  
(root@kali)~[~]  
#
```

Ejecutar un escaneo en una aplicación web

Para probar **Nikto** con una aplicación se puede usar alguna de las siguientes opciones:

1. Juice Shop (OWASP) – Aplicación con vulnerabilidades

OWASP Juice Shop es una aplicación web intencionalmente insegura diseñada para practicar pruebas de seguridad.

Instalación y ejecución de OWASP Juice Shop

```
git clone https://github.com/juice-shop/juice-shop.git
cd juice-shop
npm install
npm start
```

Juice Shop tiene muchas vulnerabilidades <http://localhost:3000>, por lo que Nikto detectará varias como:

- Exposición de cabeceras inseguras
- Cookies sin HttpOnly
- Inyecciones SQL y XSS

2. Damn Vulnerable Web Application (DVWA) – Aplicación web PHP insegura

DVWA es otra aplicación web insegura diseñada para pruebas de seguridad.

Instalación con Docker

```
docker run --rm -it -p 3000:80 vulnerables/web-dvwa
```

DVWA <http://localhost:3000> permite probar ataques como:

- SQL Injection
- Cross-Site Scripting (XSS)
- CSRF (Cross-Site Request Forgery)

3. WebGoat (OWASP) – Aplicación educativa con vulnerabilidades

WebGoat es un entorno de aprendizaje donde puedes practicar exploits.

Instalación con Docker

```
docker run --rm -it -p 3000:8080 webgoat/webgoat-8.0
```

WebGoat <http://localhost:3000/WebGoat> incluye ejercicios para probar:

- Inyección SQL
- Configuraciones inseguras
- Fallos en la autenticación

Escanear una aplicación en localhost

Para un escaneo básico en `http://localhost:3000`:

```
nikto -h http://localhost:3000
```

Nikto puede identificar varios problemas, como:

- Cookie without HttpOnly flag detected
- Server leaks version information
- Apache outdated version detected

```
(root@kali)~[/home/kali]
# nikto -h http://localhost:3000
- Nikto v2.5.0

+ Target IP: 127.0.0.1
+ Target Hostname: localhost
+ Target Port: 3000
+ Start Time: 2025-02-28 17:06:42 (GMT-5)

+ Server: No banner retrieved
+ /: Retrieved access-control-allow-origin header: *.
+ /: Uncommon header 'x-recruiting' found, with contents: /#/jobs.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /robots.txt: Entry '/ftp/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb/issues/00600600_robots-txt-file
+ /robots.txt: contains 1 entry which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/Robot_s.txt
+ assets/public/favicon.js.ico: The X-Content-Type-Options header is not set. This could allow the user agent to render the c
ontent of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnera
bilities/missing-content-type-header/
+ /localhost.pem: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /dump.tar: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /database.tar.bz2: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /dump.egg: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /database.war: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /site.pem: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /127.0.0.1.alz: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /backup.alz: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
+ /backup.tgz: Potentially interesting backup/cert file found. . See: https://cwe.mitre.org/data/definitions/530.html
```

```
(root@kali)~[/home/kali]
# nikto -h http://localhost:3000
- Nikto v2.5.0

+ Target IP: 127.0.0.1
+ Target Hostname: localhost
+ Target Port: 3000
+ Start Time: 2025-02-28 17:10:12 (GMT-5)

+ Server: Apache/2.4.25 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Head
ers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a dif
ferent fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-ty
pe-header/
+ /: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /: Cookie security created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ Root page / redirects to: login.php
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.25 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /config/: Directory indexing found.
+ /config/: Configuration information may be available remotely.
+ /docs/: Directory indexing found.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ /login.php: Admin login page/section found.
+ /.gitignore: .gitignore file found. It is possible to grasp the directory structure.
+ 7850 requests: 0 error(s) and 11 item(s) reported on remote host
+ End Time: 2025-02-28 17:10:45 (GMT-5) (33 seconds)

+ 1 host(s) tested

(root@kali)~[/home/kali]
```

```
(root@kali)-[/home/kali]
# nikto -h http://localhost:3000
- Nikto v2.5.0

+ Target IP: 127.0.0.1
+ Target Hostname: localhost
+ Target Port: 3000
+ Start Time: 2025-02-28 17:23:57 (GMT-5)

+ Server: No banner retrieved
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ 7849 requests: 0 error(s) and 2 item(s) reported on remote host
+ End Time: 2025-02-28 17:24:30 (GMT-5) (33 seconds)

+ 1 host(s) tested

(root@kali)-[/home/kali]
```

Ejecutar un análisis más profundo con Nikto

Para un escaneo más detallado en `http://localhost:3000`, se pueden usar las siguientes opciones:

1. Escaneo con detección de *plugins*, *vulnerabilidades* y *exploits*

```
nikto -h http://localhost:3000 -Tuning 123bde
```

Explicación del parámetro `-Tuning 123bde`:

- 1 - Verifica archivos interesantes (e.g., configuración, logs).
- 2 - Revisa problemas de cabeceras HTTP.
- 3 - Busca vulnerabilidades en software web común.
- b - Prueba con exploits específicos.
- d - Detecta posibles archivos de respaldo y debug.
- e - Evalúa problemas en archivos index y directorios accesibles.

2. Escaneo agresivo con detección de *archivos ocultos* y *fuzzing*

```
nikto -h http://localhost:3000 -C all
```

Explicación del parámetro `-C all`:

- `-C all` → Analiza todas las opciones de seguridad.

3. Escaneo *con autenticación* (si la aplicación tiene login)

Si la aplicación web requiere autenticación, se pueden incluir credenciales para evaluar la seguridad después de iniciar sesión:

```
nikto -h http://localhost:3000 -id usuario:contraseña
```

Ejemplo: Si la aplicación tiene una página de login, Nikto probará el escaneo después de autenticarse.

4. Escaneo con User-Agent personalizado (para evitar bloqueos)

Algunas aplicaciones bloquean ciertas herramientas de escaneo. Cambiar el User-Agent para enmascarar a Nikto:

```
nikto -h http://localhost:3000 -useragent "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
```

5. Guardar los resultados del análisis

Si se desea guardar los resultados en un archivo para su revisión posterior:

```
nikto -h http://localhost:3000 -o resultado_scan.txt
```

También se pueden exportar los resultados en formato HTML o XML:

```
nikto -h http://localhost:3000 -o resultado_scan.html -Format html  
nikto -h http://localhost:3000 -o resultado_scan.xml -Format xml
```

6. Escaneo en múltiples subdominios o direcciones IP

Si la aplicación usa varios subdominios, se pueden escanear de manera simultánea con un archivo de direcciones IP o dominios:

Ejemplo de archivo con varias direcciones IP o dominios llamado *lista_de_objetivos.txt*:

```
http://localhost:3000  
http://api.localhost:3000  
http://admin.localhost:3000
```

```
nikto -h lista_de_objetivos.txt
```

7. Escaneo con búsqueda de archivos sospechosos y puertas traseras

Nikto puede usar -Tuning para buscar archivos sospechosos:

```
nikto -h http://localhost:3000 -Tuning 45689
```

Explicación (-Tuning 45689):

- 4 - Busca archivos y directorios de backup o debug.
- 5 - Detecta posibles puertas traseras (backdoors).
- 6 - Busca inyecciones SQL y fallos de seguridad en bases de datos.
- 8 - Explora problemas en autenticación y sesiones.
- 9 - Busca configuraciones débiles en la aplicación.

8. Escaneo con bypass de firewalls y detección de tecnologías

Si la aplicación tiene mecanismos de defensa (*como WAF o IDS*), intentar este escaneo:

```
nikto -h http://localhost:3000 -useragent "Mozilla/5.0 (Windows NT 10.0; Win64; x64)" -  
useproxy http://127.0.0.1:8080
```

Explicación:

- -useragent → Enmascara el escaneo como si fuera un navegador normal.
- -useproxy → Si se usa Burp Suite o ZAP, redirige el tráfico a través del proxy en 127.0.0.1:8080.

Mitigación y Mejores Prácticas

Después de ejecutar un análisis con **Nikto**, es importante corregir las vulnerabilidades detectadas para fortalecer la seguridad de la aplicación y del servidor. A continuación, se explican con más detalle las mejores prácticas recomendadas:

* Corregir configuraciones inseguras en el servidor

Uno de los problemas más comunes en servidores web es la configuración predeterminada, que muchas veces deja expuesta información sensible o permite ataques como **clickjacking**, **cross-site scripting (XSS)** o **inyecciones**.

Implementar **cabeceras de seguridad**

Las cabeceras HTTP pueden proteger la aplicación contra diversos ataques. Algunas configuraciones recomendadas:

Ejemplo de configuración en Apache (.htaccess o httpd.conf):

```
# Evita el Clickjacking  
Header always set X-Frame-Options "DENY"  
  
# Activa la protección contra XSS  
Header always set X-XSS-Protection "1; mode=block"  
  
# Restringe el contenido embebido (CSP)  
Header always set Content-Security-Policy "default-src 'self'; script-src 'self'  
'unsafe-inline';"  
  
# Fuerza la conexión HTTPS  
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains;  
preload"
```

Habilitar el flag **HttpOnly** y **Secure** en las cookies

Las cookies sin HttpOnly y Secure pueden ser robadas a través de ataques XSS o ser transmitidas en texto plano.

Ejemplo en PHP (*php.ini* o en código PHP):

```
session_set_cookie_params([  
    'secure' => true,        // Solo enviar cookies a través de HTTPS  
    'httponly' => true,      // Evita acceso desde JavaScript  
    'samesite' => 'Strict'   // Previene ataques CSRF  
]);
```

Deshabilitar la **exposición de información** sobre el servidor y la versión

Muchos servidores muestran por defecto información sobre la versión del software, lo que facilita ataques dirigidos.

En Apache (*apache2.conf* o *.htaccess*):

```
ServerTokens Prod  
ServerSignature Off
```

* Repetir escaneos regularmente

La seguridad es un proceso continuo. Las configuraciones y actualizaciones pueden introducir nuevas vulnerabilidades, por lo que es fundamental realizar escaneos periódicos.

Automatizar pruebas con Nikto o herramientas avanzadas

Nikto puede programarse para ejecutarse automáticamente en un cronjob o script CI/CD.

Ejemplo de cronjob en Linux (ejecuta Nikto diariamente a las 2 AM):

```
0 2 * * * nikto -h http://localhost:3000 -o /var/logs/nikto_scan.log
```

Usar OWASP ZAP en Modo CLI

También puedes utilizar herramientas más avanzadas como **OWASP ZAP**, que permite análisis más completos con un proxy de interceptación. OWASP ZAP incluye su propio modo de línea de comandos. Si ya se instaló **OWASP ZAP** se puede ejecutar un escaneo rápido con:

```
zapproxy -cmd -quickurl http://localhost:3000 -quickout scan_result.html
```

Sino se dispone de la instalación, Instalar ZAP: `sudo apt update && sudo apt install zapproxy -y`

Explicación de los parámetros:

- `-cmd` → Ejecuta ZAP en modo línea de comandos.
- `-quickurl http://localhost:3000` → Escanea rápidamente el sitio.
- `-quickout scan_result.html` → Guarda el resultado en un archivo HTML.

```
(root@kali)~[/home/kali]
# zaproxy -cmd -quickurl http://localhost:3000
Found Java version 21.0.6
Available memory: 12084 MB
Using JVM args: -Xmx3021m
<?xml version="1.0"?>
<OWASPZAPReport programName="ZAP" version="2.16.0" generated="Fri, 28 Feb 2025 17:41:05">
</OWASPZAPReport>
```

Integrar DAST en el proceso de CI/CD

Para mejorar la seguridad en desarrollo, es recomendable integrar pruebas de seguridad en pipelines de CI/CD con herramientas como GitHub Actions, Jenkins o GitLab CI.

GitHub Actions con OWASP ZAP

En este caso, usaremos un escaneo **automatizado con YAML** en *GitHub Actions*.

Crear el archivo de Workflow

Crea el archivo `.github/workflows/zap_scan.yml` en tu repositorio.

```
name: DAST Scan
on: push

jobs:
  zap_scan:
    runs-on: ubuntu-latest
    steps:
      - name: Instalar OWASP ZAP
        run: sudo apt-get install zaproxy -y

      - name: Ejecutar escaneo con OWASP ZAP
        run: |
          zaproxy -cmd -autorun zap_scan.yaml

      - name: Guardar el reporte como artefacto
        uses: actions/upload-artifact@v3
        with:
          name: ZAP-Report
          path: zap_report.html
```

Crear el archivo `zap_scan.yaml` en la raíz del repositorio

Este archivo define el escaneo que OWASP ZAP ejecutará.

```
jobs:
  - type: spider
    parameters:
      url: "http://localhost:3000"
      maxDuration: 2
  - type: activeScan
    parameters:
      url: "http://localhost:3000"
```



```
recurse: true
maxDuration: 5
- type: report
  parameters:
    template: traditional-html
    reportFile: zap_report.html
    reportTitle: "ZAP Scan Report"
```

¿Qué hace este pipeline?

1. Instala OWASP ZAP en el runner de GitHub Actions.
2. Ejecuta un escaneo con Spider y Active Scan en `http://localhost:3000`.
3. Genera un reporte en HTML (`zap_report.html`).
4. Guarda el reporte como artefacto de GitHub Actions para su descarga.

Integrar ZAP en GitLab CI/CD

Si usas **GitLab CI/CD**, `.gitlab-ci.yml` sería similar a:

```
stages:
  - dast

zap_scan:
  stage: dast
  image: owasp/zap2docker-stable
  script:
    - zap.sh -cmd -autorun zap_scan.yaml
  artifacts:
    paths:
      - zap_report.html
```

Esto ejecuta ZAP dentro de un **contenedor Docker** y genera el reporte como un artefacto.

Integrar ZAP en Jenkins con un Pipeline Declarativo

Si usas **Jenkins**, agrega este pipeline en *Jenkinsfile*:

```
pipeline {
  agent any
  stages {
    stage('Download OWASP ZAP') {
      steps {
        sh 'sudo apt-get install zaproxy -y'
      }
    }
    stage('Run OWASP ZAP Scan') {
      steps {
        sh 'zaproxy -cmd -autorun zap_scan.yaml'
      }
    }
    stage('Save Report') {
      steps {
        archiveArtifacts artifacts: 'zap_report.html', fingerprint: true
      }
    }
  }
}
```